DATABASE SYSTEMS · ASSIGNMENT #4-1 · DB SCHEMA DESIGN

도서관 관리 웹 애플리케이션 데이터베이스 스키마 설계 보고서

ER Diagram, Relation Schema, 무결성 제약조건 및 추가 기능 설계

학번	이름	DBMS	구성
2022065250	문상철	PostgreSQL 18	Entity 8개 · Relation 9개 · Relationship 9개

1 설계 개요

1.1 설계 목표

이 보고서는 Assignment #4-2에서 구현할 도서관 관리 웹 애플리케이션의 데이터베이스 스키마를 설계한 결과다. 설계는 다음 세 가지를 목표로 했다.

- **기능 완전성** — 명세의 필수 기능(로그인·회원가입, 서적 검색·정렬, 대출·반납, 인기 서적 차트, 관리자의 서적·카테고리·회원 관리)과 추가 기능 4가지가 필요로 하는 데이터를 모두 표현한다.
- **정규화** — 모든 릴레이션을 BCNF로 유지해 같은 사실이 두 곳에 저장되지 않게 하고, 갱신·삽입·삭제 이상을 막는다.
- **DB 수준의 무결성** — 규칙은 가능한 한 PK·FK·UNIQUE·CHECK·부분 유일 인덱스로 DB가 직접 강제하게 하고, 여러 행을 함께 보아야 판단할 수 있는 규칙(대출 한도, 연체 여부 등)만 애플리케이션이 하나의 트랜잭션 안에서 검사한다.

1.2 스키마 구성

구분	릴레이션	담당 기능
핵심 Entity (6)	`Users` `Author` `Book` `BookCopy` `Category` `Loan`	회원·권한, 서지 정보와 실물 복본, 분류, 대출·반납 이력
교차 릴레이션 (1)	`Book_Category`	서적과 카테고리의 다대다(M:N) 관계
확장 Entity (2)	`Review` `Reservation`	추가 기능: 리뷰·별점, 예약 대기열 (및 이를 기반으로 한 통계)

1.3 핵심 설계 결정

① 서적(Book)과 실물 복본(BookCopy)의 분리

명세는 "동일한 책이 여러 권 있을 수 있고, 차트는 이를 합산하며, 대출 기록에서는 그중 정확히 어느 책을 빌렸는지 알 수 있어야 한다"고 요구한다. 그래서 제목·저자 같은 서지 정보는 `Book`에 한 번만 두고, 실물 한 권마다 `BookCopy` 한 행을 둔다. 차트는 `Book` 단위로 집계하고, 대출은 `BookCopy`를 참조한다.

② 대출(Loan)을 독립 Entity로 모델링

같은 회원이 같은 복본을 시기를 달리해 여러 번 빌릴 수 있으므로, (회원, 복본) 쌍으로는 대출 한 건을 식별할 수 없다. 따라서 대출을 관계 집합이 아닌 대리키 `loan_id`를 가진 Entity로 두고, `return_date IS NULL`인 행을 "현재 대출 중"으로 해석한다.

③ 저자(Author) 분리

저자명을 `Book`에 문자열로 반복 저장하면 표기를 고칠 때 갱신 이상이 생긴다. 서적 추가 화면도 기존 저자를 고르거나 새 저자를 입력하는 방식이고, 인기 저자 차트는 `author_id`로 집계한다. 한 책의 저자는 한 명으로 정의했다(N:1).

④ 카테고리의 다대다(M:N) 관계

"책은 동시에 여러 카테고리에 속할 수 있다"는 요구를 교차 릴레이션 `Book_Category`로 해소했다. 카테고리를 삭제하면 연결 행만 함께 지워지므로 서적은 나머지 카테고리를 그대로 유지한다.

⑤ 무결성은 먼저 DB 제약으로 보장

"대출 중인 복본은 다시 빌릴 수 없다"는 규칙은 `Loan(copy_id) WHERE return_date IS NULL` 부분 유일 인덱스로, "같은 책에 대기 중인 예약은 1건만"이라는 규칙은 `Reservation(user_id, book_id) WHERE status = 'waiting'` 부분 유일 인덱스로 강제한다. 별점 범위, 상태 값, 날짜 순서, 로그인 ID 형식은 CHECK로 막는다. 애플리케이션 버그나 동시 요청이 있어도 DB가 잘못된 상태를 거부한다.

2 Entity 설계: 역할과 필요성

각 Entity의 역할과 필요성, 그리고 릴레이션으로 옮겼을 때의 속성·타입·제약조건을 함께 정리했다. **밑줄**은 기본키(PK), *기울임*은 외래키(FK)다. 모든 Entity의 기본키는 `SERIAL` 대리키이고, 로그인 ID를 그대로 쓰는 `Users` 만 자연키를 쓴다.

Users 회원 · 관리자 계정 — 핵심

역할 일반 회원(`user`)과 관리자(`admin`) 계정의 인증 정보와 권한을 저장한다.

필요성 ID·비밀번호 로그인, 권한에 따른 기능 분리(관리자만 서적·카테고리·회원 관리), 그리고 대출·리뷰·예약을 누가 했는지 식별하는 기준이다. 관리자 가입 코드(학번 `2022065250`)는 가입 시점에만 검증하고 저장하지 않는다. 저장하면 모든 관리자 행에 같은 값이 반복될 뿐 새로운 정보가 없기 때문이다.

속성	타입	제약조건	설명
user_id	VARCHAR(20)	PK, CHECK `~ '^[A-Za-z0-9_]{3,20}$'`	로그인 ID. 화면에 그대로 출력되므로 영문·숫자·밑줄로 제한
password_hash	VARCHAR(255)	NOT NULL	salt를 포함한 scrypt 해시. 평문 비밀번호는 저장하지 않음
name	VARCHAR(50)	NULL 허용	이름 (가입 시 선택 입력)
email	VARCHAR(100)	UNIQUE, NULL 허용	이메일 (선택 입력, 입력하면 중복 불가)
role	VARCHAR(10)	NOT NULL, DEFAULT `'user'`, CHECK IN (`'user'`, `'admin'`)	권한
created_at	TIMESTAMP	NOT NULL, DEFAULT `now()`	가입 일시

Author 저자 — 핵심

역할 저자명을 한 곳에 저장하고 `Book` 이 `author_id` 로 참조한다.

필요성 저자명 중복 저장에 따른 갱신 이상을 막고, 서적 추가 시 저자 선택·신규 등록과 저자 검색·정렬, 인기 저자 차트 집계의 기준이 된다.

속성	타입	제약조건	설명
author_id	SERIAL	PK	저자 식별자
author_name	VARCHAR(100)	NOT NULL, UNIQUE	저자명. 같은 이름의 저자가 두 번 생기지 않음

Book 서적 (서지 정보) 핵심

역할 한 종류의 책에 대한 서지 정보(제목·저자·출판사·ISBN 등)를 한 행에 저장한다. 같은 책을 여러 권 소장해도 `Book` 행은 하나다.

필요성 서적 목록의 제목·저자·카테고리 표시와 검색·정렬, "같은 책 2권 동시 대출 불가" 판정, 인기 서적 차트의 집계 단위가 모두 이 Entity다.

속성	타입	제약조건	설명
book_id	SERIAL	PK	서적 식별자
isbn	VARCHAR(20)	UNIQUE, NULL 허용	국제표준도서번호 (모르면 비워 둠)
title	VARCHAR(200)	NOT NULL	제목
author_id	INTEGER	NOT NULL, FK → `Author` ON DELETE RESTRICT	저자. 책이 남아 있는 저자는 삭제 불가
publisher	VARCHAR(100)	NULL 허용	출판사
published_year	SMALLINT	NULL 허용	출판 연도
created_at	TIMESTAMP	NOT NULL, DEFAULT `now()`	등록 일시

BookCopy 실물 복본 (개별 도서) 핵심

역할 도서관이 가진 실물 책 한 권 한 권을 나타낸다. 어떤 `Book` 의 복본인지, 청구기호, 현재 상태를 저장한다.

필요성 "대출 기록에서 동일한 책 중 정확히 어느 책을 빌렸는지 확인"하려면 실물 단위의 식별자가 있어야 한다. 서적 목록의 총 수량(`COUNT(copy_id)`)과 대출 가능 수량(`status = 'available'` 인 복본 수)도 여기서 계산한다.

속성	타입	제약조건	설명
copy_id	SERIAL	PK	복본 식별자 (대출 기록에 표시되는 번호)
book_id	INTEGER	NOT NULL, FK → `Book` ON DELETE CASCADE	어느 서적의 복본인지
call_number	VARCHAR(30)	UNIQUE, NULL 허용	청구기호 (서가 위치)
status	VARCHAR(12)	NOT NULL, DEFAULT `'available'` , CHECK IN (`'available'` , `'borrowed'` , `'maintenance'`)	대출 가능 / 대출 중 / 정비 중
acquired_date	DATE	NOT NULL, DEFAULT `CURRENT_DATE`	입고일

Category 카테고리 핵심

역할 도서 분류 체계(소설, 과학, 컴퓨터 등)의 이름과 설명을 저장한다.

필요성 관리자가 서적과 무관하게 카테고리를 추가·삭제할 수 있어야 하고, 카테고리별 검색과 카테고리별 인기 차트의 기준이 된다. 서적과는 `Book_Category` 를 통해 연결된다.

속성	타입	제약조건	설명
category_id	SERIAL	PK	카테고리 식별자
category_name	VARCHAR(50)	NOT NULL, UNIQUE	카테고리명
description	VARCHAR(200)	NULL 허용	설명

Book_Category 서적–카테고리 연결 (교차 릴레이션) 핵심

역할 다대다 관계 `Belongs_To`를 릴레이션으로 옮긴 것이다. 행 하나가 "서적 b는 카테고리 c에 속한다"는 사실 하나를 나타낸다.

필요성 한 서적이 여러 카테고리에 속하려면 연결을 별도 행으로 저장해야 한다. 두 FK 모두 CASCADE이므로 서적이나 카테고리가 삭제되면 연결만 사라진다.

속성	타입	제약조건	설명
book_id	INTEGER	PK, FK → `Book` ON DELETE CASCADE	복합 기본키 (`book_id`, `category_id`)로 같은 연결의 중복을 막음
category_id	INTEGER	PK, FK → `Category` ON DELETE CASCADE	

Loan 대출 · 반납 기록 핵심

역할 한 회원이 한 복본을 빌린 대출 한 건을 기록한다. 대출일, 반납 기한, 실제 반납일을 저장하며 `return_date`가 NULL이면 아직 대출 중이다.

필요성 현재 대출 상태, 1인 최대 3권 제한, 같은 책 중복 대출 금지, 연체 판정, 회원별 대출 기록(복본 번호 포함), 최근 3개월 인기 차트가 모두 이 기록에서 나온다. 대출 기간은 7일이며, 대출 시 `due_date = loan_date + 7`로 저장한다.

속성	타입	제약조건	설명
loan_id	SERIAL	PK	대출 식별자 (반납 요청의 대상)
user_id	VARCHAR(20)	NOT NULL, FK → `Users` ON DELETE RESTRICT	대출한 회원
copy_id	INTEGER	NOT NULL, FK → `BookCopy` ON DELETE CASCADE	대출한 복본 (정확히 어느 책인지)
loan_date	DATE	NOT NULL, DEFAULT `CURRENT_DATE`	대출일
due_date	DATE	NOT NULL, CHECK (`due_date > loan_date`)	반납 기한. 이 날짜가 되면 연체
return_date	DATE	NULL 허용, CHECK (`return_date >= loan_date`)	반납일. NULL이면 대출 중

부분 유일 인덱스 `UNIQUE (copy_id) WHERE return_date IS NULL` — 한 복본에 미반납 대출이 두 건 이상 존재할 수 없다.

Review 도서 리뷰 · 별점 확장

역할 회원이 서적에 남긴 1~5점 별점과 서평을 저장한다. 리뷰는 실물 복본이 아닌 책의 내용에 대한 평가이므로 `Book`을 참조한다.

필요성 추가 기능 E1(리뷰)과 통계 S1(평점 통계)의 데이터 원천이다. `UNIQUE (user_id, book_id)`로 한 회원이 한 책에 리뷰를 하나만 남기게 해 평점 조작을 막는다.

속성	타입	제약조건	설명
review_id	SERIAL	PK	리뷰 식별자
user_id	VARCHAR(20)	NOT NULL, FK → `Users` ON DELETE RESTRICT	작성자와 대상 서적. (`user_id`, `book_id`) UNIQUE
book_id	INTEGER	NOT NULL, FK → `Book` ON DELETE CASCADE	
rating	SMALLINT	NOT NULL, CHECK (`rating BETWEEN 1 AND 5`)	별점
comment	TEXT	NULL 허용	서평
created_at	TIMESTAMP	NOT NULL, DEFAULT `now()`	작성 일시

Reservation 대출 예약 대기열 확장

역할 모든 복본이 대출 중인 서적에 회원이 걸어 둔 예약을 저장한다. 특정 복본이 아니라 "그 책 중 먼저 반납되는 아무 복본"을 기다리는 것이므로 `Book`을 참조한다.

필요성 추가 기능 E2(예약)의 대기 순서(`reserved_at`)와 상태를 관리한다. 같은 회원이 시기를 달리해 같은 책을 다시 예약할 수 있으므로 대리키를 쓰고, 대기 중인 예약의 중복만 부분 유일 인덱스로 막는다.

속성	타입	제약조건	설명
reservation_id	SERIAL	PK	예약 식별자
user_id	VARCHAR(20)	NOT NULL, FK → `Users` ON DELETE RESTRICT	예약한 회원
book_id	INTEGER	NOT NULL, FK → `Book` ON DELETE CASCADE	예약 대상 서적
status	VARCHAR(10)	NOT NULL, DEFAULT `'waiting'`, CHECK IN (`'waiting'`, `'fulfilled'`, `'cancelled'`)	대기 / 대출로 이어짐 / 취소
reserved_at	TIMESTAMP	NOT NULL, DEFAULT `now()`	예약 시각 (대기 순서의 기준)

부분 유일 인덱스 `UNIQUE (user_id, book_id) WHERE status = 'waiting'` — 한 회원이 같은 책에 대기 중인 예약을 두 건 가질 수 없다.

3 Relationship 설계: 관계와 그 이유

Entity 사이의 관계 집합은 9개다. 대응수(cardinality)와 함께 각 Entity의 참여 제약(participation)을 정의했다. **전체 참여**는 그 Entity의 모든 행이 반드시 관계에 참여해야 한다는 뜻으로, 릴레이션에서는 `NOT NULL` 외래키가 된다. 1:N 관계는 N쪽 릴레이션의 외래키로, M:N 관계는 교차 릴레이션으로 변환했다.

관계	연결 · 대응수	참여	릴레이션 변환	의미와 필요성
Has_Copy	Book **1** : **N** BookCopy	Book 전체 BookCopy 전체	`BookCopy.book_id`	한 서적은 여러 복본을 가지며, 모든 복본은 정확히 한 서적에 속한다. 동일 도서 다중 소장과 차트의 복본 합산을 함께 가능하게 한다. Book 쪽 전체 참여는 애플리케이션이 보장한다(서적 추가 시 수량 1 이상, 마지막 복본을 삭제하면 서적도 삭제).
Written_By	Book **N** : **1** Author	Book 전체 Author 부분	`Book.author_id`	모든 서적은 저자 한 명을 가지고, 한 저자는 여러 서적을 쓸 수 있다. 저자명을 한 곳에만 저장하며 인기 저자 차트의 집계 기준이 된다.
Belongs_To	Book **M** : **N** Category	Book 부분 Category 부분	`Book_Category`	한 서적은 여러 카테고리에, 한 카테고리에는 여러 서적이 속한다. 관리자가 빈 카테고리를 먼저 만들 수 있으므로 양쪽 모두 부분 참여다.
Borrows	Users **1** : **N** Loan	Users 부분 Loan 전체	`Loan.user_id`	회원 한 명이 여러 건을 대출한다. 대출 한도(3권), 연체 여부, 회원별 대출 기록을 회원 단위로 조회하는 근거다.
Borrowed_In	BookCopy **1** : **N** Loan	BookCopy 부분 Loan 전체	`Loan.copy_id`	한 복본은 시기를 달리해 여러 번 대출된다. 대출이 `Book`이 아닌 `BookCopy`와 연결되므로 같은 책 중 정확히 어느 복본을 빌렸는지 기록된다.
Writes	Users **1** : **N** Review	Users 부분 Review 전체	`Review.user_id`	회원은 여러 서적에 리뷰를 남길 수 있다. 리뷰 작성 자격(해당 서적 대출 이력)을 확인하는 기준이다.

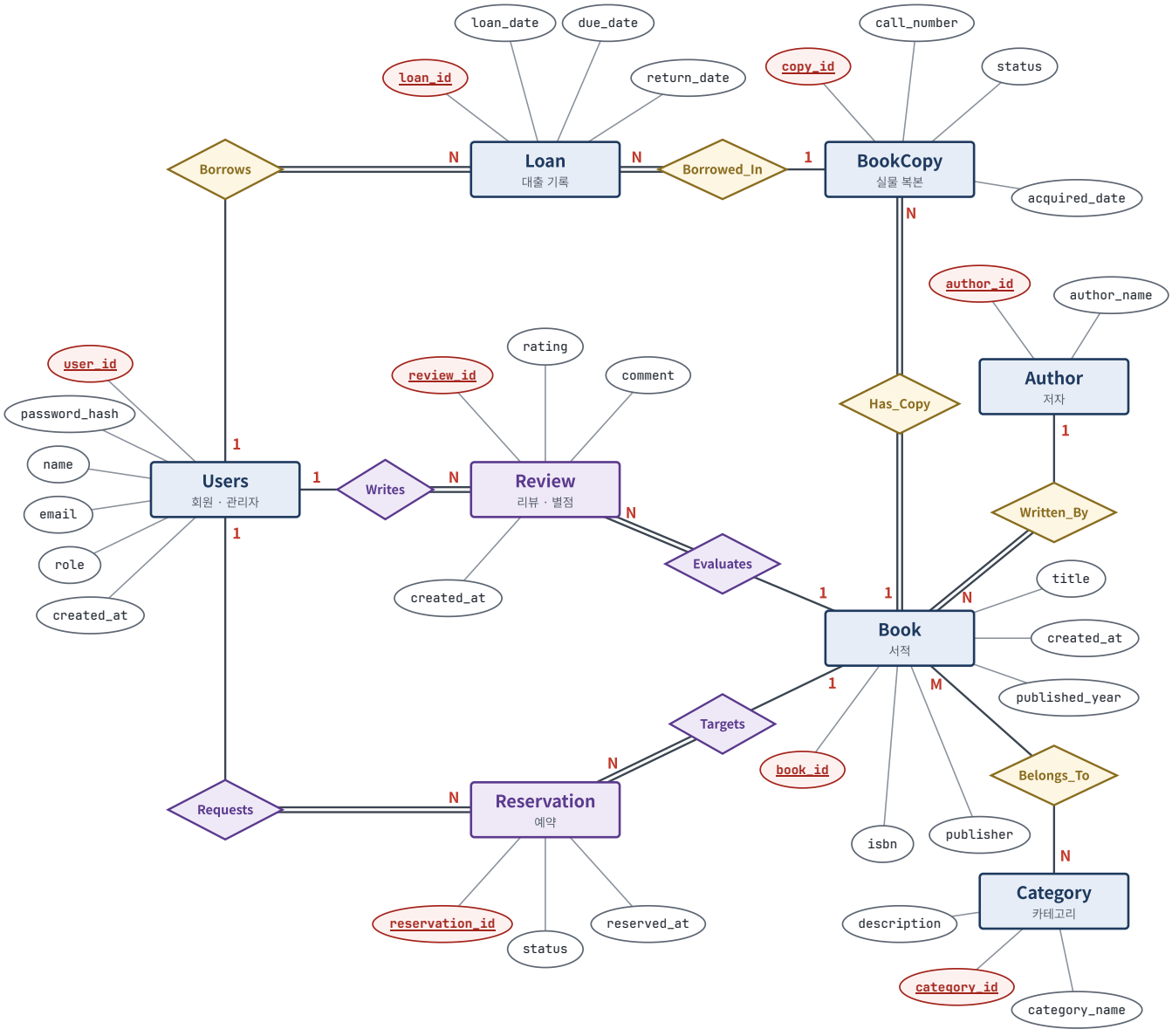
관계	연결 · 대응수	참여	릴레이션 변환	의미와 필요성
Evaluates	Book **1** : **N** Review	Book 부분 Review 전체	`Review.book_id`	한 서적은 여러 회원의 리뷰를 받는다. 서적별·카테고리별 평점 통계의 집계 경로다.
Requests	Users **1** : **N** Reservation	Users 부분 Reservation 전체	`Reservation.user_id`	회원은 여러 서적을 예약할 수 있다. 내 예약 목록과 예약 취소의 기준이다.
Targets	Book **1** : **N** Reservation	Book 부분 Reservation 전체	`Reservation.book_id`	한 서적에 여러 예약이 쌓이며, `reserved_at` 순서가 대기 순번이 된다.

Loan · Review · Reservation을 Entity로 둔 이유. 세 가지 모두 두 Entity를 잇는 관계처럼 보이지만, 관계 집합의 한 원소는 참여 Entity의 키 조합으로 식별되어야 한다. 같은 회원이 같은 복본을 여러 번 빌리고(Loan), 같은 책을 취소 후 다시 예약하므로(Reservation) 키 조합만으로는 한 건을 구분할 수 없다. 그래서 각각 대리키를 가진 Entity로 모델링하고 양쪽과 1:N 관계를 맺게 했다. Review는 (회원, 서적) 조합이 유일하지만, 다른 두 이력 Entity와 구조를 맞추고 리뷰 수정·삭제 요청을 `review_id` 하나로 가리키기 위해 같은 방식을 택했다.

4 ER Diagram

Chen 표기법으로 그린 전체 ER Diagram이다. 남색 Entity는 필수 기능, 보라색 Entity와 관계는 추가 기능(리뷰·예약)에 해당한다. 교차 릴레이션 `Book_Category` 는 릴레이션 변환의 결과이므로 ER Diagram에서는 M:N 관계 `Belongs_To` 로만 나타난다.

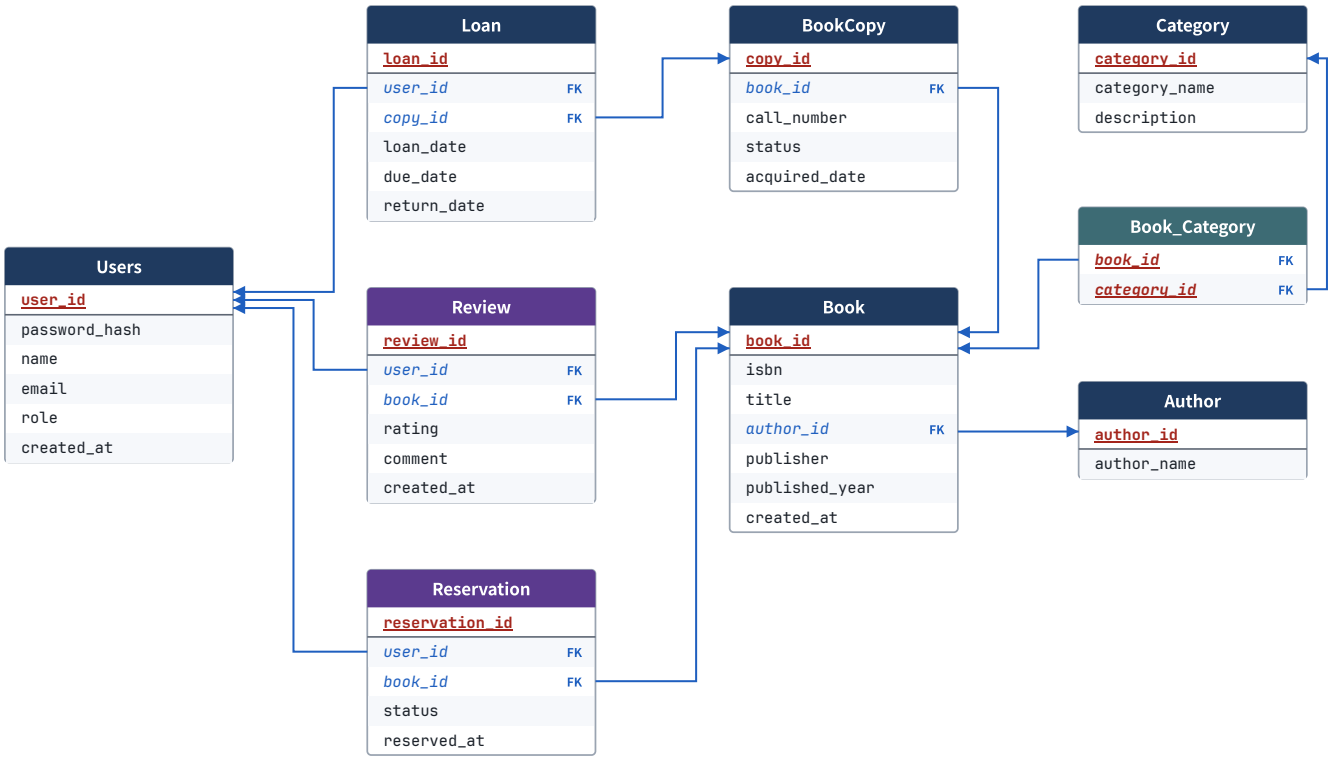
직사각형 Entity 집합 **마름모** Relationship 집합 **타원** 속성 (밑줄: 기본키) **1 · N · M** 대응수 **이중선** 전체 참여 / **단일선** 부분 참여



[그림 1] 도서관 관리 시스템 ER Diagram (Chen 표기법)

5 Relation Schema

ER Diagram을 관계형 모델로 변환한 결과다. 각 상자는 릴레이션 하나이고, 구분선 위의 **밑줄** 속성이 기본키, 화살표는 외래키(*FK*)가 참조하는 기본키를 가리킨다. 1:N 관계 8개는 N쪽 외래키로, M:N 관계 `Belongs_To` 는 교차 릴레이션 `Book_Category` 로 변환되어 릴레이션은 모두 9개다.



[그림 2] 도서관 관리 시스템 Relation Schema Diagram

5.1 릴레이션 스키마 표기

`Users`	(user_id, password_hash, name, email, role, created_at)
`Author`	(author_id, author_name)
`Book`	(book_id, isbn, title, *author_id*, publisher, published_year, created_at)
`BookCopy`	(copy_id, *book_id*, call_number, status, acquired_date)
`Category`	(category_id, category_name, description)
`Book_Category`	(*book_id*, *category_id*)
`Loan`	(loan_id, *user_id*, *copy_id*, loan_date, due_date, return_date)
`Review`	(review_id, *user_id*, *book_id*, rating, comment, created_at)
`Reservation`	(reservation_id, *user_id*, *book_id*, status, reserved_at)

5.2 참조 무결성: 외래키와 삭제 규칙

삭제 규칙은 "부모가 사라지면 자식이 의미를 잃는가"를 기준으로 정했다. 의미를 잃는 종속 데이터는 `CASCADE` , 이력 보존이 중요한 데이터의 부모는 `RESTRICT` 로 삭제를 막는다.

외래키	ON DELETE	이유
`Book.author_id → Author`	RESTRICT	책이 남아 있는 저자가 지워지면 서적이 저자를 잃는다.
`BookCopy.book_id → Book`	CASCADE	서적이 삭제되면 그 복본도 존재 의미가 없다.
`Book_Category.* → Book, Category`	CASCADE	연결 행만 지워지고, 서적은 나머지 카테고리를 유지한다.
`Loan.copy_id → BookCopy`	CASCADE	관리자가 복본을 제거하면 그 복본의 대출 기록도 함께 제거된다. 대출 중인 복본은 애플리케이션이 삭제를 거부한다.
`Review.book_id`, `Reservation.book_id → Book`	CASCADE	서적이 없어지면 리뷰와 예약도 의미가 없다.
`Loan` · `Review` · `Reservation.user_id → Users`	RESTRICT	회원 탈퇴는 이번 범위에 없으며, 이력이 있는 회원은 삭제할 수 없게 해 기록을 보존한다.

5.3 도메인 · 키 제약

대상	제약	보장하는 규칙
`Users.user_id`	CHECK `~ '^[A-Za-z0-9_]{3,20}$'`	로그인 ID 형식. 화면에 그대로 출력되는 값이라 안전한 문자만 허용
`Users.role`	CHECK IN (`'user'`, `'admin'`)	권한 값은 두 가지뿐
`Users.email`, `Author.author_name`, `Category.category_name`, `Book.isbn`, `BookCopy.call_number`	UNIQUE	후보키의 중복 방지
`BookCopy.status`	CHECK IN (`'available'`, `'borrowed'`, `'maintenance'`)	복본 상태 값 제한
`Loan`	CHECK (`due_date > loan_date`), CHECK (`return_date >= loan_date`)	날짜 순서
`Loan(copy_id)`	부분 유일 인덱스 `WHERE return_date IS NULL`	한 복본에 미반납 대출은 최대 1건 (이중 대출 방지)
`Review`	UNIQUE (`user_id`, `book_id`), CHECK (`rating BETWEEN 1 AND 5`)	회원당 서적별 리뷰 1개, 별점 범위
`Reservation`	CHECK `status` IN (`'waiting'`, `'fulfilled'`, `'cancelled'`), 부분 유일 인덱스 (`user_id`, `book_id`) `WHERE status = 'waiting'`	상태 값 제한, 같은 책에 대기 중인 예약 중복 방지

6 필수 기능과 스키마의 대응

명세의 필수 기능이 스키마의 어느 부분으로 뒷받침되는지, 그리고 규칙을 DB 제약과 애플리케이션 트랜잭션 중 어디에서 보장하는지 정리했다.

요구사항	사용하는 스키마	보장 방법
ID·비밀번호 로그인, 회원/관리자 구분	`Users` (`user_id`, `password_hash`, `role`)	해시 비교로 인증. `role` CHECK로 권한 값 제한
관리자 가입 시 추가 코드(학번)	`Users.role`	가입 요청의 코드가 `2022065250`일 때만 `role = 'admin'` 으로 INSERT (코드는 저장하지 않음)

요구사항	사용하는 스키마	보장 방법
서적 목록: 제목·저자·카테고리·수량·대출 가능 여부	`Book` ⋈ `Author`, `Book_Category` ⋈ `Category`, `BookCopy`	카테고리는 `string_agg`로 묶고, 총 수량은 복본 수, 대출 가능 수량은 `status = 'available'`인 복본 수로 계산
제목·저자·카테고리로 정렬 및 검색	`Book.title`, `Author.author_name`, `Category.category_name`	검색어는 `ILIKE` 바인딩 파라미터. 카테고리로 검색해도 그 서적의 카테고리는 모두 표시되도록 `EXISTS` 조건으로 거름
대출 중이 아닌 책만 대출	`BookCopy.status`, `Loan.return_date`	트랜잭션에서 복본 행을 `FOR UPDATE`로 잠그고 `'available'` 확인. 부분 유일 인덱스가 이중 대출을 최종 차단
동시에 최대 3권	`Loan`	해당 회원의 미반납 대출이 3건 미만인지 트랜잭션에서 검사
같은 책 2권 동시 대출 불가	`Loan` ⋈ `BookCopy`	회원의 미반납 대출 중 같은 `book_id`가 없는지 검사 (복본이 달라도 같은 책이면 거부)
연체 회원 대출 불가	`Loan.due_date`	미반납이면서 `CURRENT_DATE >= due_date`인 대출(7일 이상 대출)이 하나라도 있으면 거부
반납 (본인 대출만)	`Loan`, `BookCopy.status`	`loan_id`와 세션 회원이 모두 일치하고 미반납인 행만 갱신, 같은 트랜잭션에서 복본을 `'available'`로 되돌림
회원별 대출 기록, 정확한 복본 확인	`Loan.copy_id`	대출 기록마다 복본 번호(`copy_id`)와 서적·저자를 함께 표시
최근 3개월 인기 서적·저자 차트 (전체 / 카테고리별, 복본 합산)	`Loan` → `BookCopy` → `Book` (→ `Book_Category`)	`loan_date`로 3개월을 거른 뒤 `book_id` (저자 차트는 `author_id`)로 GROUP BY하므로 복본별 대출이 합산됨
서적 추가·제거	`Book`, `BookCopy`, `Author`, `Category`, `Book_Category`	추가는 한 트랜잭션에서 새 저자·카테고리 등록, 서적 1행, 복본 N행, 연결 행 삽입. 제거는 복본 단위이며 대출 중이면 거부하고, 마지막 복본이 제거되면 서적도 삭제
카테고리 추가·제거, 다중 카테고리	`Category`, `Book_Category`	카테고리 삭제 시 연결 행만 CASCADE로 삭제되어 서적은 다른 카테고리를 유지
회원 목록 열람 (관리자)	`Users`	ID 또는 권한으로 검색. 비밀번호 해시는 조회하지 않음

6.1 대출 트랜잭션

대출 규칙은 여러 행을 함께 보아야 하므로 하나의 트랜잭션에서 검사한다. 복본 행을 먼저 잠가 같은 복본에 대한 동시 요청을 직렬화한다.

```
BEGIN;
SELECT book_id, status FROM BookCopy WHERE copy_id = $1 FOR UPDATE;   -- status = 'available' 이어야 함

SELECT COUNT(*)                                          AS open_loans,    -- 3 미만
       COUNT(*) FILTER (WHERE CURRENT_DATE >= l.due_date) AS overdue,       -- 0 이어야 함
       COUNT(*) FILTER (WHERE c.book_id = $3)             AS same_book      -- 0 이어야 함
  FROM Loan l JOIN BookCopy c ON c.copy_id = l.copy_id
 WHERE l.user_id = $2 AND l.return_date IS NULL;

-- 대기 중인 예약이 있으면 가장 먼저 예약한 회원만 대출 가능 (추가 기능 E2)
INSERT INTO Loan (user_id, copy_id, loan_date, due_date)
VALUES ($2, $1, CURRENT_DATE, CURRENT_DATE + 7);
UPDATE BookCopy SET status = 'borrowed' WHERE copy_id = $1;
COMMIT;
```

6.2 최근 3개월 인기 서적 차트

```
SELECT b.title, a.author_name, COUNT(*) AS borrow_count
  FROM Loan l
  JOIN BookCopy c ON c.copy_id   = l.copy_id
  JOIN Book b     ON b.book_id   = c.book_id
  JOIN Author a   ON a.author_id = b.author_id
 WHERE l.loan_date >= CURRENT_DATE - INTERVAL '3 months'
-- 카테고리별 차트는 여기에 조건 하나를 더한다:
--   AND EXISTS (SELECT 1 FROM Book_Category bc
--               WHERE bc.book_id = b.book_id AND bc.category_id = $1)
 GROUP BY b.book_id, b.title, a.author_name
 ORDER BY borrow_count DESC
 LIMIT 10;
```

7 추가 기능 설계

명세는 "명세 외 기능 3가지 이상, 그중 통계 기능은 정확히 2가지"를 요구한다. 이에 따라 비통계 기능 2가지(E1, E2)와 통계 기능 2가지(S1, S2), 모두 4가지를 설계했다. 통계가 2가지를 넘지 않도록, 이 두 기능 외의 화면에는 평균·순위 같은 집계 지표를 추가하지 않는다. 인기 서적·저자 차트는 필수 기능이므로 여기에 포함하지 않는다.

E1 도서 리뷰 및 별점

비통계 기능 · `Review`

목적 대출 횟수만으로는 알 수 없는 책의 내용과 난이도에 대한 평가를 회원끼리 공유한다.

동작 서적 상세 페이지에 리뷰 목록과 평균 별점을 보여 준다. 그 서적을 한 번이라도 대출한 회원만 리뷰를 쓸 수 있고(`Loan` ⋈ `BookCopy` 로 확인), 회원당 서적별 리뷰는 하나다. 작성자는 자기 리뷰를 수정·삭제할 수 있다.

스키마 `Review` 의 `UNIQUE (user_id, book_id)` 와 `CHECK (rating BETWEEN 1 AND 5)` 가 중복 리뷰와 잘못된 별점을 막는다.

E2 대출 예약 대기열

비통계 기능 · `Reservation`

목적 모든 복본이 대출 중인 인기 도서를 기다리는 회원에게 순서를 보장해, 반납 시점을 노려 계속 새로고침해야 하는 불편을 없앤다.

동작 대출 가능한 복본이 없고, 그 책을 대출 중이 아니며, 대기 중인 예약도 없을 때 예약할 수 있다. 대기 예약이 있는 동안 반납된 복본은 가장 먼저 예약한 회원(`reserved_at` 최솟값)만 대출할 수 있고, 그 회원이 대출하면 예약은 `fulfilled` 가 된다. 회원은 자기 예약을 취소(`cancelled`)할 수 있다.

스키마 부분 유일 인덱스 `(user_id, book_id) WHERE status = 'waiting'` 가 중복 대기를 막고, 대기 순번은 `reserved_at` 으로 정한다.

S1 서적 · 카테고리별 평점 통계

통계 기능 1/2 · `Review` `Book` `Book_Category` `Category`

목적 인기 차트가 보여 주는 양적 지표(대출 횟수)를 질적 지표(만족도)로 보완한다.

내용 ① 서적별 평균 별점과 리뷰 수 ② 카테고리별 평균 별점, 리뷰 수, 그리고 가장 평점이 높은 서적. 별도의 통계 테이블 없이 `Review` 에서 매번 집계하므로 원본 데이터와 항상 일치한다.

```
SELECT c.category_name,
       COUNT(r.review_id)      AS review_count,
       ROUND(AVG(r.rating), 2) AS avg_rating
  FROM Category c
  JOIN Book_Category bc ON bc.category_id = c.category_id
  LEFT JOIN Review r    ON r.book_id      = bc.book_id
 GROUP BY c.category_id, c.category_name
 ORDER BY avg_rating DESC NULLS LAST;
```

S2 **월별 대출 · 연체 통계와 이용자 독서 통계** 통계 기능 2/2 · `Loan` `Users`

목적 관리자에게는 도서관 운영 지표를, 회원에게는 자신의 독서 습관을 보여 준다.

내용 ① 월별 대출 건수와 연체율 (관리자) ② 회원별 누적 대출 수 순위, 즉 다독 순위 (다른 회원의 기록이 드러나므로 관리자 전용) ③ 본인의 누적 대출 수와 기한 내 반납률 (모든 회원). 반납일이 기한 이후이거나(`return_date >= due_date`), 아직 반납하지 않았는데 기한이 지난 대출을 연체로 본다.

```
SELECT to_char(loan_date, 'YYYY-MM') AS month,
       COUNT(*)                       AS loans,
       ROUND(100.0 * COUNT(*) FILTER (WHERE COALESCE(return_date, CURRENT_DATE) >= due_date)
             / COUNT(*), 1)           AS overdue_rate_pct
  FROM Loan
 GROUP BY month
 ORDER BY month DESC;
```

8 설계 검토: 정규화와 트레이드오프

8.1 정규화

모든 릴레이션에서 비자명 함수 종속의 결정자는 후보키뿐이므로 BCNF를 만족한다. 예를 들어 `Book` 에서는 `book_id` 와 후보키 `isbn` 만 다른 속성을 결정하고, 저자명은 `Author` 로 분리되어 `author_id → author_name` 같은 이행 종속이 남지 않는다. `Book_Category` 는 모든 속성이 키인 릴레이션이다.

`Loan.due_date` 는 현재 규칙으로는 `loan_date + 7` 이지만, 대출 시점의 정책으로 정해진 기한이라는 별개의 사실로 보아 저장한다. 대출 기간 정책이 바뀌거나 기한을 연장해도 과거 기록의 기한은 그대로여야 하므로 `loan_date → due_date` 는 성립하는 종속이 아니다. 그래서 CHECK도 등식이 아닌 `due_date > loan_date` 로 두었다.

8.2 의도적인 트레이드오프

결정	얻는 것	대가와 대응
`BookCopy.status` 유지	`'maintenance'` 처럼 `Loan` 으로 표현할 수 없는 상태를 담고, 서적 목록의 대출 가능 수량을 `Loan` 조인 없이 계산한다.	`'borrowed'` 여부는 미반납 `Loan` 과 중복된다. 대출·반납 트랜잭션에서 두 테이블을 함께 갱신하고, 부분 유일 인덱스가 이중 대출을 최종 차단한다.
복본 삭제 시 대출 기록 CASCADE	명세의 "서적 제거"를 그대로 구현하고, 마지막 복본이 사라지면 서적까지 정리된다.	삭제된 복본의 과거 대출 기록도 사라지고 지난 차트 집계에서 빠진다. 대출 중인 복본은 삭제를 거부한다. 이력 보존이 필요해지면 `status = 'retired'` 같은 논리 삭제로 바꿀 수 있다.
`user_id` 자연키	로그인 ID를 바꾸는 기능이 없어 키가 안정적이고, 외래키 값만으로 회원을 알아볼 수 있다.	정수 대리키보다 비교 비용이 크지만 이 규모에서는 무시할 수 있다. 형식을 CHECK로 제한해 화면 출력 시에도 안전하다.
리뷰 자격을 애플리케이션에서 검사	"대출 이력이 있는 회원만 리뷰 가능"을 별도 테이블 없이 기존 `Loan` 으로 판단한다.	FK나 CHECK로는 다른 테이블의 이력을 조건으로 걸 수 없으므로, 리뷰 작성 요청마다 트랜잭션 안에서 확인한다.